Viva Questions and Answers
Software Development I (4COSC006C)

01. Difference between = and == in Python

In Python, = and == serve different purposes:

= (Assignment Operator):

The = operator is used for assignment in Python.

It is used to assign the value on the right side to the variable on the left side.

== (Equality Operator):

The == operator is used for comparison in Python.

It checks if the values on both sides are equal.

It returns True if the values are equal and False otherwise.

In summary, = is used for assignment, while == is used for equality comparison. It is a common mistake to confuse these two operators, so it is important to use them correctly to avoid unintended behavior in your code.

02. Difference between ‘print’ and ‘return’ in Python?

print:

The print statement is used to display information on the console or terminal.

It is primarily used for debugging purposes or for providing information to the user.

The print statement does not affect the flow of the program; it simply outputs the specified information.

return:

The return statement is used to exit a function and return a value to the calling code.

It is used to pass the result of the function back to the caller.

The return statement terminates the execution of the function.

In summary, print is used for displaying information, typically for debugging or user output, whereas return is used to exit a function and provide a result to the calling code. Functions in Python can use both print and return statements, depending on the desired behavior.

03. Difference between lists and dictionaries in Python

List:

Lists are ordered collections of items, meaning that the order in which elements are added to the list is preserved.

Elements in a list can be accessed by their index.

Lists are mutable, which means you can change their content by adding or removing elements.

Lists are created using square brackets [].

Dictionary:

Dictionaries are unordered collections of items. In a dictionary, items are stored as key-value pairs.

The order of items in a dictionary is not guaranteed.

Dictionaries are mutable, so you can add, remove, or modify key-value pairs.

Each item in a dictionary consists of a key and its associated value.

Keys are unique within a dictionary, and they are used to access values.

Dictionaries are created using curly braces {}.

In summary, a list is an ordered and mutable collection of elements, while a dictionary is an unordered and mutable collection of key-value pairs. The choice between them depends on the specific requirements of your data and how you plan to access and manipulate it.

04. What is the user defined function in Python?

A user-defined function in Python is a function that is created by the programmer, as opposed to built-in functions that are provided by the Python language. These functions allow you to group a set of statements into a reusable block of code, which can then be called multiple times from various parts of your program.

05. Different between if, elif and else in Python

In Python, if, elif (else if), and else are used in conjunction to create conditional statements for decision-making. These constructs are part of the control flow mechanisms in Python and allow you to execute different blocks of code based on different conditions.

- if is used for the initial condition.
- elif is used for additional conditions to be checked if the previous conditions are false.
- else is used for a default action when none of the preceding conditions are true.

06. Can we catch an exception for syntax error?

In Python, syntax errors cannot be caught using exceptions because they occur during the parsing phase of the Python interpreter, before the code is executed. Syntax errors indicate that the code is not written according to the correct syntax rules of the Python language.

When a syntax error is encountered, the Python interpreter raises a Syntax Error exception, and the program execution is halted. The error message typically provides information about the location and nature of the syntax error.

Because syntax errors occur before the code is executed, they cannot be caught using try-except blocks or other exception-handling mechanisms. The best way to address syntax errors is to carefully review and correct the code to conform to the proper syntax rules.

On the other hand, runtime errors, such as those that occur during the execution of a program, can be caught using try-except blocks to handle exceptions and prevent the program from crashing.

07. What is the difference between a list and a tuple in Python?

Mutability:

List: Lists are mutable, meaning you can change their contents by adding or removing elements, or by modifying existing elements.

Tuple: Tuples are immutable, and once they are created, their elements cannot be modified, added, or removed. If you need a constant collection, a tuple is a good choice.

Syntax:

List: Created using square brackets [].

Tuple: Created using parentheses ().

Performance:

List: Because of mutability, lists generally have more overhead than tuples in terms of memory and processing time.

Tuple: Tuples are more memory-efficient and can provide faster iteration compared to lists.

Use Cases:

List: Use lists when you have a collection of items that may need to be modified, and you want to take advantage of list methods like append(), remove(), and extend().

Tuple: Use tuples when you have a collection of items that should remain constant throughout the program, and you want to ensure their immutability.

Methods:

List: Lists have more built-in methods for various operations, such as append(), remove(), sort(), etc.

Tuple: Tuples have fewer methods compared to lists because of their immutability.

08. How can you handle exceptions in Python?

Handling exceptions in Python is done using the try, except, else, and finally blocks. Here's an explanation of how each block works in the context of exception handling:

try Block:

The try block contains the code that might raise an exception.

Python attempts to execute the code within this block.

except Block:

If an exception occurs in the try block, control is transferred to the corresponding except block.

The except block specifies the type of exception it can handle. If the exception matches the specified type, the code within the except block is executed.

Multiple except blocks can be used to handle different types of exceptions.

else Block:

The else block, if present, is executed if no exceptions are raised in the try block.

It is optional and typically contains code that should run when no exceptions occur.

finally Block:

The finally block, if present, is executed regardless of whether an exception occurs or not.

It is often used for cleanup operations that should be performed, such as closing files or releasing resources.

09. How can you open and read from a file in Python?

Opening a File:

Use the open() function to open a file. The function takes two arguments: the file name and the mode.

Modes include:

'r': Read (default mode).

'w': Write (creates a new file or truncates an existing file).

'a': Append (opens the file for writing, and appends to the end of the file).

'b': Binary mode (e.g., 'rb' or 'wb').

After opening, the open() function returns a file object.

Reading from a File:

Once the file is open, you can use various methods to read its contents.

Common methods include:

read(): Reads the entire file as a single string.

readline(): Reads a single line from the file.

readlines(): Reads all lines in the file into a list.

If you're reading a large file, it's often more efficient to read it line by line using a loop.

Closing the File:

It's good practice to close the file after performing the necessary operations.

Using the with statement automatically closes the file when the block is exited.

10. What is the purpose of the pass statement in Python?

In Python, the pass statement is a null operation, and it serves as a placeholder where syntactically some code is required but no action is desired or necessary. The pass statement does nothing and is essentially a no-op. It is often used when the syntax requires a statement, but you want to convey that no action should be taken.

11. Explain the purpose of the with statement in Python.

The with statement in Python is used to simplify resource management, particularly when working with external resources like files, network connections, or database connections. It ensures that certain operations are properly set up and torn down, even if an exception occurs during the execution of the code inside the with block. The primary purpose of the with statement is to provide a clean and convenient way to work with context managers.

12. What is the purpose of the __str__ method in Python classes?

In Python, the __str__ method is a special method that is called by the str() built-in function and the print() function. Its purpose is to return a human-readable string representation of an object. By defining the __str__ method in a class, you can customize how instances of that class are displayed when converted to a string.

13. How can you install external packages in Python using pip?

In Python, pip is the package installer that allows you to install external packages from the Python Package Index (PyPI). Here are the basic steps to install external packages using pip

14. What is the Global Keyword in Python?

When you want to explicitly declare a variable as global within a function (when you are modifying a global variable inside a function), you use the global keyword.

15. How can you reverse a list in Python?.

In Python, you can reverse a list using the reverse() method or by using slicing.

16. How can you convert a string to lowercase in Python?

In Python, you can convert a string to lowercase using the lower() method. The lower() method is a built-in string method that returns a new string with all alphabetic characters converted to lowercase.

17. Explain the purpose of the zip() function in Python.

The zip() function in Python is used to combine multiple iterable objects (such as lists, tuples, or strings) into a single iterator of tuples. Each tuple contains elements from the input iterables at the same index. The purpose of zip() is to create an iterator that aggregates elements from corresponding positions in the input iterables.

18. How can you find the maximum and minimum values in a list in Python?

In Python, you can find the maximum and minimum values in a list using the built-in functions max() and min()

19. Explain the difference between except Exception as e and except Exception, e in exception handling.

In Python, the except clause is used for exception handling. When catching exceptions, there are two common forms:

except Exception as e:

except Exception, e:

However, it's important to note that the second form (except Exception, e:) is deprecated and is not valid syntax in Python 3. In Python 3, only the first form (except Exception as e:) is used.

20. How can you read the contents of a file line by line in Python?

In Python, you can read the contents of a file line by line using a for loop or the readline() method.

21. Compare and contrast the readline() and readlines() methods.

Comparison:

Behavior: readline() reads one line at a time, while readlines() reads all lines at once and returns a list.

Memory Usage: readline() is more memory-efficient for large files because it reads and processes one line at a time. readlines() reads the entire file into memory, which can be memory-intensive for large files.

Iteration: With readline(), you can iterate over lines in a file using a loop. With readlines(), you iterate over the list of lines.

Newline Characters: Both methods include newline characters ('\n') in their results.

Use Cases:

Use readline() when you want to process lines one at a time or when dealing with large files where memory efficiency is crucial.

Use readlines() when you need to access all lines at once and want them stored in a list.

It's important to choose the method that best fits your specific requirements based on the size of the file and how you plan to process its contents.

22. Define what a Python module is.

A module is a file containing Python code that defines variables, functions, and classes. The primary purpose of a module is to organize code into reusable units. Modules help in structuring large programs and facilitate code reuse and maintenance.

23. Explain different ways to import modules in Python.

there are several ways to import modules, each offering different levels of control over the imported names and allowing for various naming conventions. Here are the primary ways to import modules

1. Simple Import:

Import the entire module and use its functions or variables using the module name.

2. Import with Alias:

Import a module with an alias (a different name) to make the code more concise.

3. Import Specific Names:

Import specific functions, variables, or classes directly into the current namespace.

4.Import All Names (Not Recommended):

Import all names from a module into the current namespace. This is generally not recommended to avoid name clashes.

When you use the statement from module_name import *, it imports all names from the specified module into the current namespace. This means that all functions, variables, and classes defined in the module become directly accessible without the need to prefix them with the module name. While this approach might seem convenient at first, it is generally not recommended for several reasons:

Namespace Pollution:

Importing all names into the current namespace can lead to namespace pollution. This means that names from the imported module may clash with names in the current code, causing unintended and potentially problematic behavior.

Readability and Maintenance:

It becomes unclear which names are actually being used in your code, especially when dealing with large modules. This can make the code less readable and harder to maintain.

Potential Name Conflicts:

There is a risk of name conflicts with existing names in your code. If the module contains names that overlap with names used in your program, it can lead to bugs that are difficult to identify and fix.

Code Clarity:

Explicitly importing only the names you need (using import module_name or from module_name import specific_name) enhances code clarity. It makes it clear to readers which names are being used in a particular module or script.

5. Import with Alias for Specific Names:

Import specific names with aliases for better readability.

6. Conditional Import:

Import a module conditionally based on a certain condition.

7. Importing Submodules:

Import specific submodules from a module.

8. Importing All Names from Submodule:

Import all names from a submodule.

24. What are the parameters and arguments in python?

In Python, parameters and arguments are terms used in the context of functions to describe the variables that are used to pass information into and out of functions. Here are the definitions for each:

Parameters:

Parameters are the variables that are used in the function definition.

They act as placeholders for the values that a function will receive when it is called.

Parameters are defined in the parentheses of a function definition.

Arguments:

Arguments are the actual values that are passed to a function when it is called.

They correspond to the parameters of the function and provide the actual data that the function will operate on.

Arguments are passed in the parentheses of a function call.

It's important to note that the terms "parameter" and "argument" are often used interchangeably, but the key distinction is that parameters are used in the function definition, and arguments are the actual values provided when calling the function.

In addition, Python supports different types of parameters, such as positional parameters, keyword parameters, default parameters, and variable-length parameters (*args and **kwargs), which provide flexibility in how functions are defined and called.

25. Provide an example of using pop() and remove() methods with lists.

Certainly! Both pop() and remove() are methods used with lists in Python, but they serve different purposes.

Using pop() Method:

The pop() method removes and returns the element at a specified index. If no index is provided, it removes and returns the last element by default.

Using remove() Method:

The remove() method removes the first occurrence of a specified value from the list. If the value is not found, it raises a ValueError.

In this example, pop() is used to remove elements by index, while remove() is used to remove elements by value. Remember that pop() modifies the list in place and returns the removed element, while remove() modifies the list but does not return any value.

Choose between pop() and remove() based on whether you want to remove elements by index or by value.

26. What is the purpose of the pass statement in Python?

the pass statement is a null operation or a no-op. It serves as a placeholder where syntactically some code is required but no action needs to be taken or no functionality is desired. The pass statement does nothing and is often used as a stub or a placeholder to indicate that a block of code is intentionally left empty.

Using pass in these scenarios allows the code to remain syntactically correct while indicating that the specific block is intentionally left blank or needs further development. It helps in writing code incrementally, especially when creating outlines or scaffolding for future implementation.

27. How can you create an infinite loop using a while loop?

Creating an infinite loop using a while loop involves setting a condition that is always True. Since the condition is always true, the loop will continue to execute indefinitely until it is manually interrupted or the program is terminated.

28. What is the purpose of the break statement in a while loop?

The break statement in a while loop is used to prematurely exit the loop based on a specified condition. When the break statement is encountered, the control immediately exits the loop, and the program continues with the next statement after the loop.

The primary purpose of the break statement in a while loop is to provide a way to terminate the loop before its natural completion based on some specific condition. This can be useful in scenarios where you want to exit the loop when a certain condition is met, rather than waiting for the loop to finish all iterations.

In this example, the while True creates an infinite loop. However, the break statement is used to exit the loop when the count variable reaches 5. Without the break statement, the loop would continue indefinitely. The break statement provides a way to control the flow of the loop based on a specific condition, making the loop more flexible and responsive to dynamic situations.

29. Explain the purpose of the continue statement in a while loop.

The continue statement in a while loop is used to skip the rest of the code inside the loop for the current iteration and move on to the next iteration. When the continue statement is encountered, the control immediately goes back to the beginning of the loop, bypassing any code that follows the continue statement within the current iteration.

The primary purpose of the continue statement is to alter the flow of the loop based on a specific condition, allowing certain iterations to be skipped while continuing with the next ones.

In this example, the while loop runs until count reaches 5. However, when count is equal to 3, the continue statement is encountered, and the rest of the code inside the loop for that iteration is skipped. The loop then continues with the next iteration.

The continue statement is useful when you want to skip certain iterations of a loop based on a specific condition without exiting the loop entirely. It provides a way to control the flow of the loop and tailor its behavior according to the requirements of your program.

30. Explain exit() function in Python?

In Python, the exit() function is a built-in function that is used to terminate the execution of the program. When exit() is called, it raises the SystemExit exception, which, if not caught, will terminate the program.

While exit() is useful in certain scenarios, it's generally recommended to use it cautiously. In many cases, it's better to let exceptions propagate naturally, allowing the program to exit with an appropriate exit code.

For normal termination, it's common to let the program exit without explicitly calling exit(). When the main script or program completes its execution, it will automatically exit.

Keep in mind that if you're working in an interactive environment like the Python REPL or an IPython notebook, calling exit() will terminate the entire Python process, including the interactive session. Use with caution to avoid unintended consequences.